

Shrinking and Stretching of Images

Brandon Marquez Salazar

1 Objective and Approach

In the current work, the objective was to modify an existing algorithm to achieve independent shrinking and stretching of a given image along its width and height. The starting point was the in-class program 07, which uses a single scaling factor f . This original algorithm handles two cases: $f > 1$ for enlargement and $f < 1$ for reduction.

The original algorithm is as follows:

Algorithm 1 Image Scaling (Escalar)

```
1: procedure Escalar( $I_{in}, I_{out}, f$ )
2:   if  $f > 1.0$  then                                     ▷ Upscaling: nearest neighbor
3:      $s \leftarrow f, I_{out}.create(h \cdot s, w \cdot s)$ 
4:     for  $u \in [0, h)$  do
5:       for  $v \in [0, w)$  do
6:          $p \leftarrow I_{in}[u, v]$ 
7:         for  $i \in [u \cdot s, (u + 1) \cdot s)$  do
8:           for  $j \in [v \cdot s, (v + 1) \cdot s)$  do
9:              $I_{out}[i, j] \leftarrow p$ 
10:          end for
11:        end for
12:      end for
13:   else                                                    ▷ Downscaling: average pooling
14:      $s \leftarrow \lfloor 1/f \rfloor$ 
15:     if  $h \not\equiv 0 \pmod s$  OR  $w \not\equiv 0 \pmod s$  then
16:        $I_{out} \leftarrow I_{in}$                                 ▷ Dimension error
17:     else
18:        $I_{out}.create(h/s, w/s)$ 
19:       for  $u \in [0, h)$  step  $s$  do
20:         for  $v \in [0, w)$  step  $s$  do
21:            $I_{out}[u/s, v/s] \leftarrow \frac{1}{s^2} \sum_{i=u}^{u+s-1} \sum_{j=v}^{v+s-1} I_{in}[i, j]$ 
22:         end for
23:       end for
24:       for  $i \in [0, h)$  do
25:          $AjusteDeRango(I_{out})$ 
26:       end for
27:     end if
28:    $I_{out}.convertTo(CV_8UC1)$ 
29: end procedure
```

2 Proposed Algorithm

The proposed new algorithm expands the original logic to accommodate two independent scaling factors, f_x and f_y , for the x-axis (width) and y-axis (height), respectively.

The proposed algorithm is:

Algorithm 2 Asymmetric Image Resize

```

1: procedure Resize( $f_x, f_y$ )
2:    $I_{in} \leftarrow \text{this} \rightarrow \text{image}$  ▷ Input image  $I_{in}[u, v]$ 
3:    $(s_x, s_y) \leftarrow \begin{cases} (f_x, f_y) & \text{if } f_x > 1, f_y > 1 \\ (\lfloor 1/f_x \rfloor, f_y) & \text{if } f_x < 1, f_y > 1 \\ (f_x, \lfloor 1/f_y \rfloor) & \text{if } f_x > 1, f_y < 1 \\ (\lfloor 1/f_x \rfloor, \lfloor 1/f_y \rfloor) & \text{otherwise} \end{cases}$ 
4:   Output dimensions:  $(h_{out}, w_{out}) \leftarrow \begin{cases} (h \cdot s_y, w \cdot s_x) & \text{if } f_x > 1, f_y > 1 \\ (h \cdot s_y, w/s_x) & \text{if } f_x < 1, f_y > 1 \\ (h/s_y, w \cdot s_x) & \text{if } f_x > 1, f_y < 1 \\ (h/s_y, w/s_x) & \text{otherwise} \end{cases}$ 
5:   if  $\exists$  downscaling AND  $(w \not\equiv 0 \pmod{s_x} \text{ OR } h \not\equiv 0 \pmod{s_y})$  then
6:     return error
7:   end if
8:   for  $(i, j) \in [0, h_{out}) \times [0, w_{out})$  do
9:      $(u, v) \leftarrow \text{MapCoordinates}(i, j, s_x, s_y, f_x, f_y)$ 
10:     $I_{out}[i, j] \leftarrow \text{ComputePixel}(I_{in}, u, v, s_x, s_y, f_x, f_y)$ 
11:   end for
12:   if  $f_x < 1$  AND  $f_y < 1$  then AjusteDeRango( $I_{out}$ )
13:   end if
14:    $\text{this} \rightarrow \text{image} \leftarrow I_{out}$ 
15: end procedure

```

Algorithm 3 Pixel Coordinate Mapping

```

1: function MapCoordinates( $i, j, s_x, s_y, f_x, f_y$ )
2:   if  $f_x > 1$  AND  $f_y > 1$  then return  $(i/s_y, j/s_x)$ 
3:   end if
4:   if  $f_x < 1$  AND  $f_y > 1$  then return  $(i/s_y, j \cdot s_x)$ 
5:   end if
6:   if  $f_x > 1$  AND  $f_y < 1$  then return  $(i \cdot s_y, j/s_x)$ 
7:   end if
8:   if  $f_x < 1$  AND  $f_y < 1$  then return  $(i \cdot s_y, j \cdot s_x)$ 
9:   end if
10: end function

```

This expansion from one to four primary cases (combinations of $f_x \leq 1$ and $f_y \leq 1$) introduced significant complexity to the original program structure. The core challenge was managing the independent scaling along each dimension.

3 Results and Implementation Issues

Unfortunately, the implemented code demonstrated several problems in its output. These artifacts are likely attributable to the construction and pixel-access logic of the new image matrix, which proved highly sensitive to the independent handling of the two scaling factors.

Algorithm 4 Pixel Computation

```
1: function ComputePixel( $I_{in}$ ,  $u$ ,  $v$ ,  $s_x$ ,  $s_y$ ,  $f_x$ ,  $f_y$ )
2:   if  $f_x > 1$  AND  $f_y > 1$  then
3:     return  $I_{in}[[u], [v]]$                                 ▷ Nearest neighbor
4:   end if
5:   if  $f_x < 1$  AND  $f_y > 1$  then
6:      $sum \leftarrow \sum_{y=u}^{u+s_y-1} \sum_{x=v}^{v+s_x-1} I_{in}[y, x]$ 
7:     return  $sum / (s_x \cdot s_y)$                                 ▷ Average pooling in x
8:   end if
9:   if  $f_x > 1$  AND  $f_y < 1$  then
10:     $sum \leftarrow \sum_{y=u}^{u+s_y-1} \sum_{x=v}^{v+s_x-1} I_{in}[y, x]$ 
11:    return  $sum / (s_x \cdot s_y)$                                 ▷ Average pooling in y
12:   end if
13:   if  $f_x < 1$  AND  $f_y < 1$  then
14:     $sum \leftarrow \sum_{y=u}^{u+s_y-1} \sum_{x=v}^{v+s_x-1} I_{in}[y, x]$ 
15:    return  $sum / (s_x \cdot s_y)$                                 ▷ Average pooling both
16:   end if
17: end function
```

The program output is as follows:

```
1  g++ code/tareas/tarea02/main.cxx code/src/ProcesamientoDigitalImagenes.cxx `pkg-
   config --cflags --libs opencv4` -lm -g -o code/tareas/tarea02/exe
2  cd code/tareas/tarea02 && ./exe
3  Nombre:      ../../res/testImg/C01.bmp
4  Número de filas:  480
5  Número de columnas: 640
6  Total de pixeles:  307200
7  Número de canales:  1
8  Intensidad máxima:  nan
9  Intensidad mínima:  nan
10 Nombre:      ../../res/testImg/C02.bmp
11 Número de filas:  0
12 Número de columnas: 0
13 Total de pixeles:  0
14 Número de canales:  1
15 Intensidad máxima:  nan
16 Intensidad mínima:  nan
17 Nombre:      ../../res/testImg/Family.png
18 Número de filas:  1200
19 Número de columnas: 1600
20 Total de pixeles:  1920000
21 Número de canales:  4
22 Intensidad máxima:  nan
23 Intensidad mínima:  nan
24 terminate called after throwing an instance of 'cv::Exception'
25   what():  OpenCV(4.12.0) ../../modules/highgui/src/window.cpp:973: error: (-215:
   Assertion failed) size.width>0 && size.height>0 in function 'imshow'
26
27 Aborted (core dumped)
28 make: *** [Makefile:52: tarea02] Error 134
```

4 Conclusion and Future Work

Due to a lack of time, the resolution of these errors is deferred to a future effort. The primary conclusion of this work is that the complexity of this two-factor scaling approach requires meticulous attention to the image matrix creation and pixel access patterns. Future work will focus on correcting the identified implementation flaws.